UNIVERSIDAD DE SAN BUENAVENTURA

Guide #3 – Multilinear Regression

Introduction

The aim of this exercise is to implement the algorithm to calculate the hypothesis and model for a multilinear regression dataset.

To get started with this exercise, there will be different files that you have to complete to finish this task. Throughout the exercise, you will execute the script *Lab3.py* which calls functions that you will modify; please do not modify any other file, just follow the instructions of the assignment.

Files included.

Lab3.py – *main script to execute the practice #1*
[*] multiGradient.py – *File to compute the gradient descent.*
[*] reading.py – *File to read the plain file of data.*
[*] Scaling.py – *File to scale the dataset.*
Plotting.py – Function to plot the data.

* Indicates files you will need to complete.

1. Data reading function

In this part, the idea is to implement multilinear regression to predict the price of a diamond, according to different features such as carat, depth, and its dimensions (x,y,z), label as x_1, x_2, x_3, x_4 ,x_5 in the learning model.

carat (0.2--5.01): The carat is the diamond's physical weight measured in metric carats. One carat equals 1/5 gram and is subdivided into 100 points. Carat weight is the most objective grade of the 4Cs.

Dimensions:

x length in mm (0--10.74)
y width in mm (0--58.9)
z depth in mm (0--31.8)

depth total depth percentage = z / mean(x, y) = 2 * z / (x + y) (43--79): The depth of the diamond is its height (in millimetres) measured from the culet (bottom tip) to the table (flat, top surface).

The file *diamonds1.csv* contains the dataset for the multilinear regression algorithm. The headers in the first row, expose the order of the columns included in the dataset. Complete the

function ***data()*** in the file "***reading.py***" to load the dataset in Python, return *X* with the data of the carat, depth, 'x', 'y', 'z' and the output *Y* with the data of the '*price*'.

After it is completed, it can be seen on the terminal:

```
Tamaño X: (7999, 5), tipo:<class 'numpy.ndarray'>
Tamaño Y: (7999, 1), tipo:<class 'numpy.ndarray'>
First 5 examples of the dataset
[[2.30e-01 6.15e+01 3.95e+00 3.98e+00 2.43e+00 3.26e+02]
[2.10e-01 5.98e+01 3.89e+00 3.84e+00 2.31e+00 3.26e+02]
[2.30e-01 5.69e+01 4.05e+00 4.07e+00 2.31e+00 3.27e+02]
[2.90e-01 6.24e+01 4.20e+00 4.23e+00 2.63e+00 3.34e+02]
[3.10e-01 6.33e+01 4.34e+00 4.35e+00 2.75e+00 3.35e+02]]
```

2. Feature Scaling (Z-score Normalization)

Once the data is completely organized, it is possible to see that the feature *z* is 1000 times greater than the value of *carat*. When its features differ in the order of magnitude, performing feature scaling helps gradient descent to converge even quicker.

In the file Scaling.py, complete ***FeatureScaling( )*** to get the mean value of each feature from the dataset and additionally scale the value by its respective standard deviation. (numpy already has these two functions inside its library, use *help(numpy)* to find the corresponding method to use). Finally return the 3 variables designated in the *return* of the algorithm.

Note: Remember each column in the matrix X corresponds to one feature.

3. Vectorized Cost Function

In a previous Lab, the algorithm for gradient descent was implemented on a univariate regression problem, with the difference that this problem includes five features. The hypothesis and the gradient algorithm remain with the same coding rules.

To use your previous cost function algorithm in the gradient descent implementation, be sure that it supports multiple variables by implementing a vectorized solution and code your new algorithm in ***cost( )***, a function included in the file *multigradient.py.*

$$f_{(w,b)} = \begin{bmatrix} b \\ \omega_1 \\ \omega_2 \\ \vdots \\ \omega_n \end{bmatrix} \cdot \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}^T = b + \omega_1 x_1 + \omega_2 x_2 + \cdots + \omega_n x_n$$

4. Gradient Descent algorithm

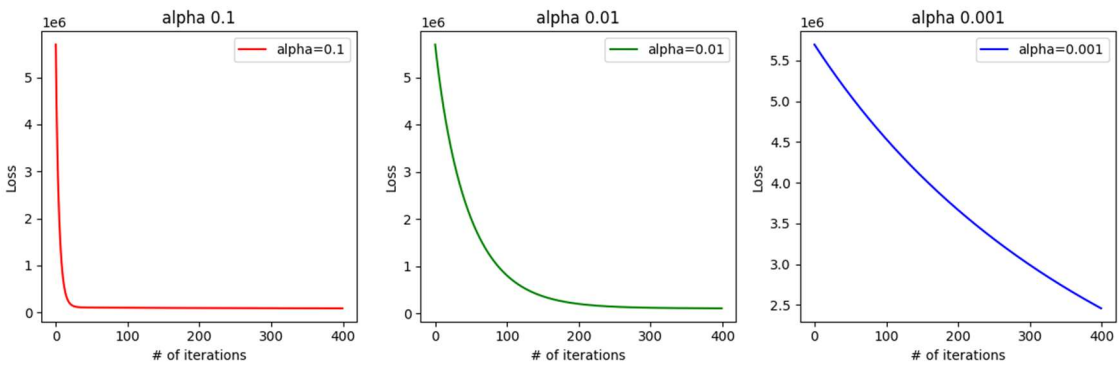
Then to run the gradient descent algorithm, it is needed to select a proper learning rate. For this new phase of your algorithm, complete the gradient descent algorithm for multivariate

functions. Besides, you should store the value of the cost function *J* after each iteration, to know the variation of the loss according to the different chosen learning rates.

Note: remember the value of the first column of w should be 1.

$$\omega_n := \omega_n - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{(x_{(i)})} - y_{(i)} \right) \cdot x_{n(i)}$$

After implementing correctly your gradient descent algorithm, it will be gotten a figure as follows: (you can change the value of alpha in the main file "line 31" to see how it affects the loss).



5. Prediction for multivariate model

With a well-chosen learning rate, run the algorithm to predict the price of diamond with:

***carat**=0.76; **depth**=61.8; **x**=5.82; **y**=5.86; **z**=3.61;*

Remember to scale the features of prediction too.

Guide #4 – Normal equation

Introduction

The aim of this exercise is to implement the algorithm to implement the normal equation in a multilinear regression dataset.

To get started with this exercise, there will be different files that you have to complete to finish this task. Throughout the exercise, you will execute the script *Lab4.py* which calls functions that you will modify; **please do not modify any other file, just follow the instructions of the assignment.**

Programming Ex. 3: Multilinear Regression

Machine Learning

Supervised Algorithms

UNIVERSIDAD DE SAN BUENAVENTURA

Files included.

Lab4.py – *main script to execute the practice #4*
[*] reading.py – *Function to read the plain file of data.*
[*] n_Equ.py – *Function to compute the normal equation.*
* Indicates files you will need to complete.

1. Data reading function

In this part, the idea is to implement multilinear regression to predict the price of a diamond, according to different features such as carat, depth, and its dimensions (x,y,z), label as x_1, x_2, x_3, x_4 ,x_5 in the learning model.

The file *diamonds1.csv* contains the dataset for the multilinear regression algorithm. The headers in the first row expose the order of the columns included in the dataset. Complete the function ***data()*** in the file "***reading.py***" to load the dataset in Python, return *X* with the data of the carat, depth, 'x', 'y', 'z' and the output *Y* with the data of the '*price*'. *(same as the previous exercise)*

2. Normal equation

After the data is completely organized, compute the normal equation as shown:

$$\omega = (X^T \cdot X)^{-1} \cdot (X^T \cdot y)$$

In the file *n_Equ.py,* complete the function **normalEqu*( )*** to compute the normal equation for the multilinear regression problem on the dataset ***data2.txt***. Remember it does not require to scale the features, as well as it does not require to iterate over the data. It is still needed to add a column of 1's to get the intercept term ***b***.

3. Prediction

Once the values of w were found, use it to predict the price of a diamond with the same characteristics as the previous exercise.

carat*=0.76;* ***depth****=61.8;* ***x****=5.82;* ***y****=5.86;* ***z****=3.61;*

Remember not to scale the features of prediction too for the normal equation.